

Shot Probability Prediction Built on Gradient Boosting

The model is trained off two different seasons worth of shots, totaling ~425,000 non-fouled shots. The approach was to split the data separately into the seasons, as after a discovery on both the testing and training sets where training set contained 2 seasons and testing set had a whole new different season. The main idea is to ensure the model works off this approach of one season predicting the next one. This meant using a leave-one-season-out split instead of a random k-fold, since a random split would mix shots from the same season into both training and validation and report a score better than the model would actually achieve on an unseen season. The model uses Python's LightGBM package, a gradient boosting decision tree model. The result shows an average of ~162 trees created and reveals a log-loss score of 0.6258 against a baseline of 0.6897.

A deeper look inside the raw datasets set the trajectory for building the model with a few key discoveries. The first being the 29 columns that were given, where more than half were strings to identify the players, teams, and seasons. Alongside the shot physicals, gamestate was stated such as transition, halfcourt, or direct second chance. Gamestate and shottype helped reveal the reason for the shot clock spike on both extremes of 0 and 24, where in the NBA a team is given a 24 second shot clock once the ball is inbounded and is reset when the ball touches the rim or backboard where many shots would be a tip in. The dataset was also set up to be clean in terms of all shots being mirrored to one half of the court and missing values being structural which were the distances of contesters due to shot varying in having 1-4 contesters. The shot clock spikes were tested to ensure none were shot clock errors, distance of contesters was backed up by number of contesters column, and all shots being marked on one half of NBA court within the coordinates of $\{(0,0),(0,25),(0,-25),(-47,25),(-47,-25),(-41.75,0)$ [Rim Center]}. This led to a quick visualization of the shots on the court, where the shots were spread out similar to what is expected for a team's seasons worth of shots. The majority of shots were taken either near the rim or in the paint and all along the three-point arc shown in Figure 1-A. Make rate was also shown to be what is expected where make rate dropped with shots taken long distance and a high make rate when at the hoop with the rest of shots on the court falling in between a make rate of 40-50%.

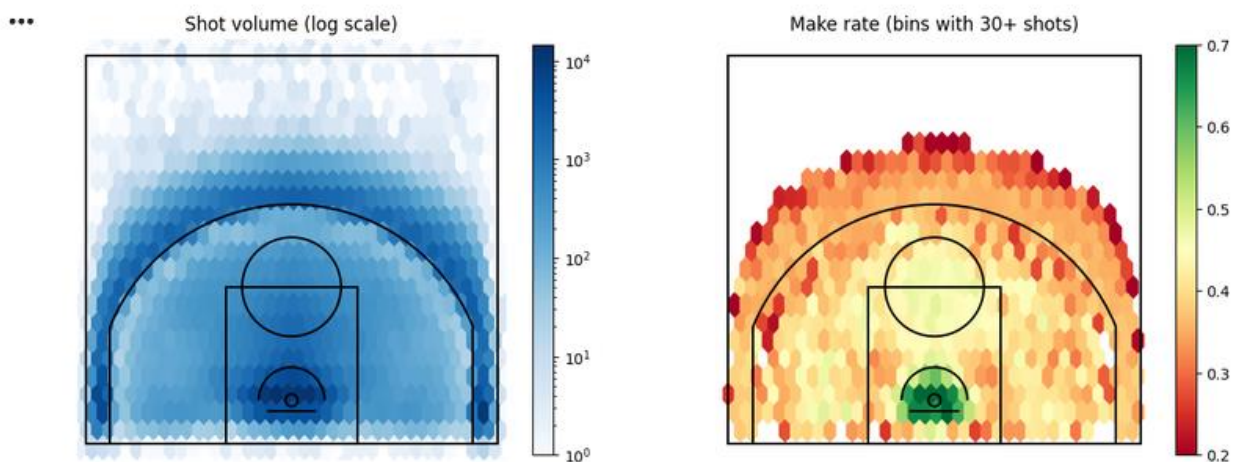


Figure 1-A

Further charts were generated to view the spread of the shots in figure 2-A, with more than half the shots taken at the three point line or at the rim, mid-range shots were very low in comparison which is more due to the state of NBA gameplans in today's game. Shots over the shot clock interval (Figure 2-B) showed an even spread with a clean curve and a peak at 13-14 seconds close to half of the 24 second shot clock. Make rate at 23-24 seconds was higher than the rest of the intervals which is due to the type of shot that can be taken at that interval such as putbacks or tips. Shot types were also consistent with the modern game, with over half of the shots being jumpers, which was noted to be a big part of the approaches that were taken later in the data modeling process (Figure 2-C). It was also noted from the Num_contesters chart (Figure 2-D) that the shots were mainly taken with only 1 defender, again pointing to the practices of the game where gameplans are set to find the best shot meaning shots with 2 or more defenders are understood to be of very low shot quality.

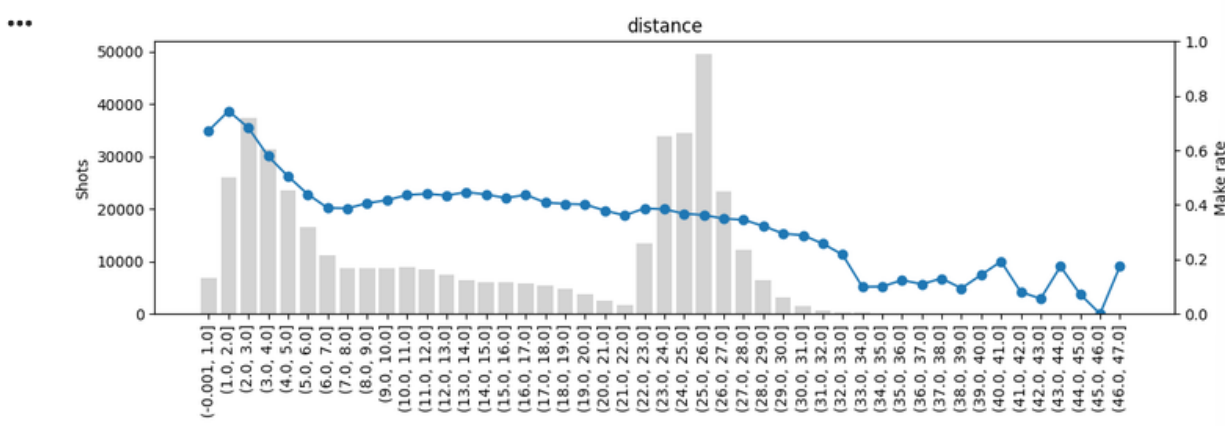


Figure 2-A

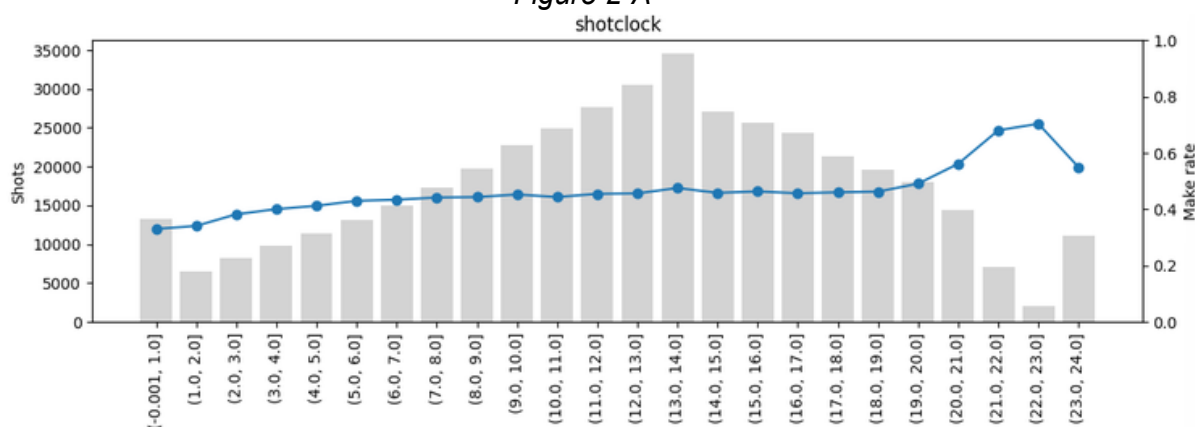


Figure 2-B

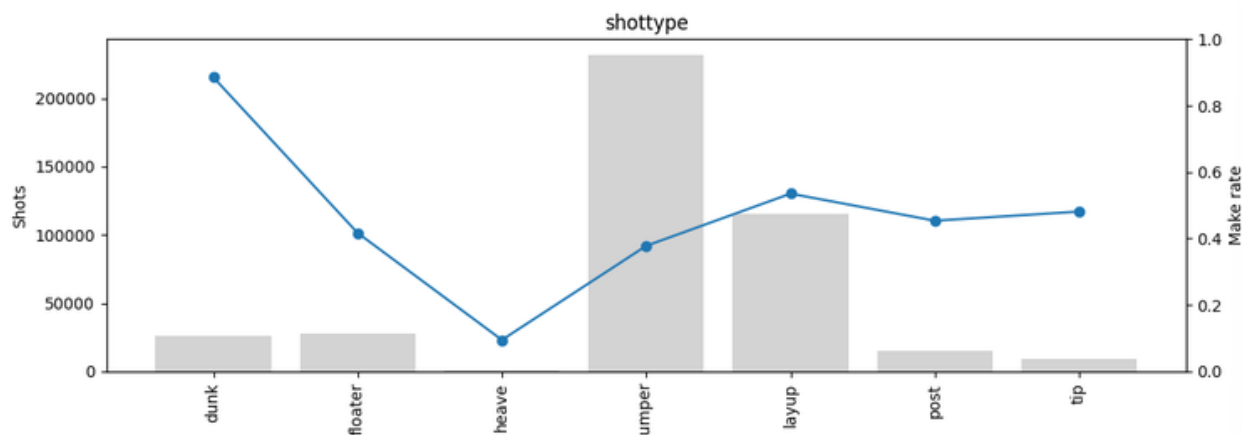


Figure 2-C

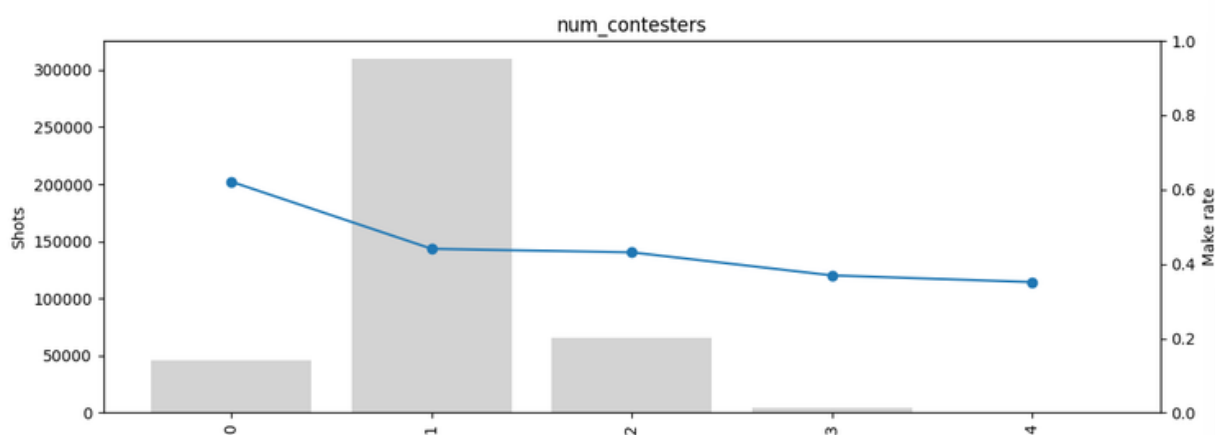


Figure 2-D

The features selected in the final model were broken down into 4 categories; core shot, location, defender geometry both defender location and motion, and shooter. The core shot group consisted of the basic shot 'physicals' which are what makes up the quality of the shot, like distance, shot type, and shotclock. The context of the core shot is the basis of the shot's probability as in a NBA game jumpers, layups, putbacks are always judged by number of defenders, the game state and speed of it, which are straightforward metrics in what you want in a shot which would be space and time. An important part of the core shot metrics was that shot distance was capped at 35 feet which clips the tail end of long range shots or shots that are not quality shots someone wants to take but more a heave or a throw up, which just minimizes the noise on this front.

What this project leaned towards was the 'physical' difficulty of the shot versus the revealed judgment, which is what the other players believe about the shot quality. Revealed judgement includes the defender closing behavior, where defender's behavior says a lot, such as a hard close in because it's open or it may be due needing to closely contest a player who is seen as a shooter. Whether the closest defender is contesting is also key, as it shows what a defender believes about that specific shot such as too far out, or unusual shot for a player taking it and indirectly tells you if the shot was open regarding no contest. Again these differ depending on the type of shot, due to the amount of jumpers that are taken in today's game and in the data set the features look to revolve around the jumper abstractly.

Following the same idea is where defender geometry is introduced, for example the defender delta which is the ground covered by the closest defender in the final second and this is what drives the idea of a hard close in or not. The defender distance was clipped at 20 and log-transformed, which takes on the same shape as the shot distance curve as a defender standing 2 feet away versus 4 feet changes the quality of the shot enormously and defender distance at either 15 or 17 feet don't make a difference in the shot at all which is why log1p compresses the far end so the real splits go where they change the shot. Defender geometry breaks into 2 groupings, the contesting spacing and closing motion. In contesting spacing, the features used are derived from the dataset that was given, [cont_min] measures the distance to the nearest contesting player which is the best pressure metric in the model at 8.1% gain in feature importance. The distinction between the contesting and the defender matters here, because the contesting player is the player actually challenging the shot and therefore the one affecting shot quality. The feature [cont_mean] then averages the distance across all contesting players, working on top of the [cont_min] because it then catches whether help is collapsing, where one defender 3 feet away with nobody near returns a high mean, and three defenders converging returns much lower, in theory these affect the shot but in different ways. The same grouping also contains features [n_within_4] and [n_within_6], measures the count of contesting players inside those radii, assisting the model by giving it crossing points it would otherwise have to infer, where 4 feet is approximately arm's length away and 6 feet is close enough to affect a shooter without needing to contest.

The second group in the defender geometry block is closing motion, are 5 features that are all derived from [closestdefapproach] the original attribute that captures a defender's distance 4 times in a 1 second snapshot, with [closestdefdist] closing defender distance containing final distance of the defender at release. [def_start] is the defender distance at 1.0 second out, meaning where the defender began, and it is included because it is mechanically tied to [def_delta], the total ground covered by a defender over the full second, where negative means a close in, and a positive return means the defender peeled off. The delta here is the strongest point in terms of revealed judgement because this metric infers what the defender's intention was, which is either closing in hard on the shot to affect the quality or peeling off because the shot is bad for the specific player taking the shot. The [def_late] is the ground covered in the final 0.25 seconds before shot release, the idea is that a defender can close four feet early and then stop giving a different shot than one from still arriving as the ball leaves. Due to having the data from the defender over a 1 second window, [def_accel] defender acceleration can be interpreted and reveals whether a defender is rushing into a contest or if they have their feet set which are two different interactions for a shot contest. The [close_rate_vs_speed] closing rate vs the shooter speed represents the ground covered divided by the shooter's own speed, it is based on the relative motion where a three feet closing against a set shot differs from a defender moving the same three feet but with a shooter moving at 12ft/s reveals a defender failing to keep up.

The feature importance (figure 3-A) shows what the model relied on for the features with defender geometry having its fair share with the biggest metric involving the category is it brought log-loss down by 0.0017, the same as the next category; location. The location category logs the location of the shot on the court, which felt was important to explicitly define because the idea here is similar to that of hot zones, but on a broader scale. The feature [abs_y] absolute y is how far the shot was taken from the center line of the court, and the [angle] feature gives

the direction of the shot relative to the rim, so together they place the shot exactly on the floor in a way that raw distance alone cannot. A [corner_3] flag was introduced here as well which gives the direct signal if it was a corner three or not, it turned out to be a redundant feature, shown by the 0.0% model importance in the feature importance chart (Figure 3-A), because [abs_y] is continuous and the model can place its own split wherever a corner three begins rather than relying on my flag. Finally a shooter category was created and had many iterations of it leaving only one feature [shooter_vol] the shooter volume. The conventional approach was a season window make rate for a player, it was rejected because make rate in a season creates more noise for the model than anything as the objective was to find the quality of the shot, not if a player usually makes a three. A count instead of the player shots contains no outcome information and therefore cannot leak within the data set training , the shooter volume reveals whether this is a player the offense routinely shoots with, without needing to define skill, where unseen players get a 0 and is honestly able to describe them as low usage.

Each of these blocks contributed its share to log-loss, with defender geometry and location tested to show they each brought non-overlapping information, a key signal that a shot in a game has many more variables in it than a shooter's raw ability alone.

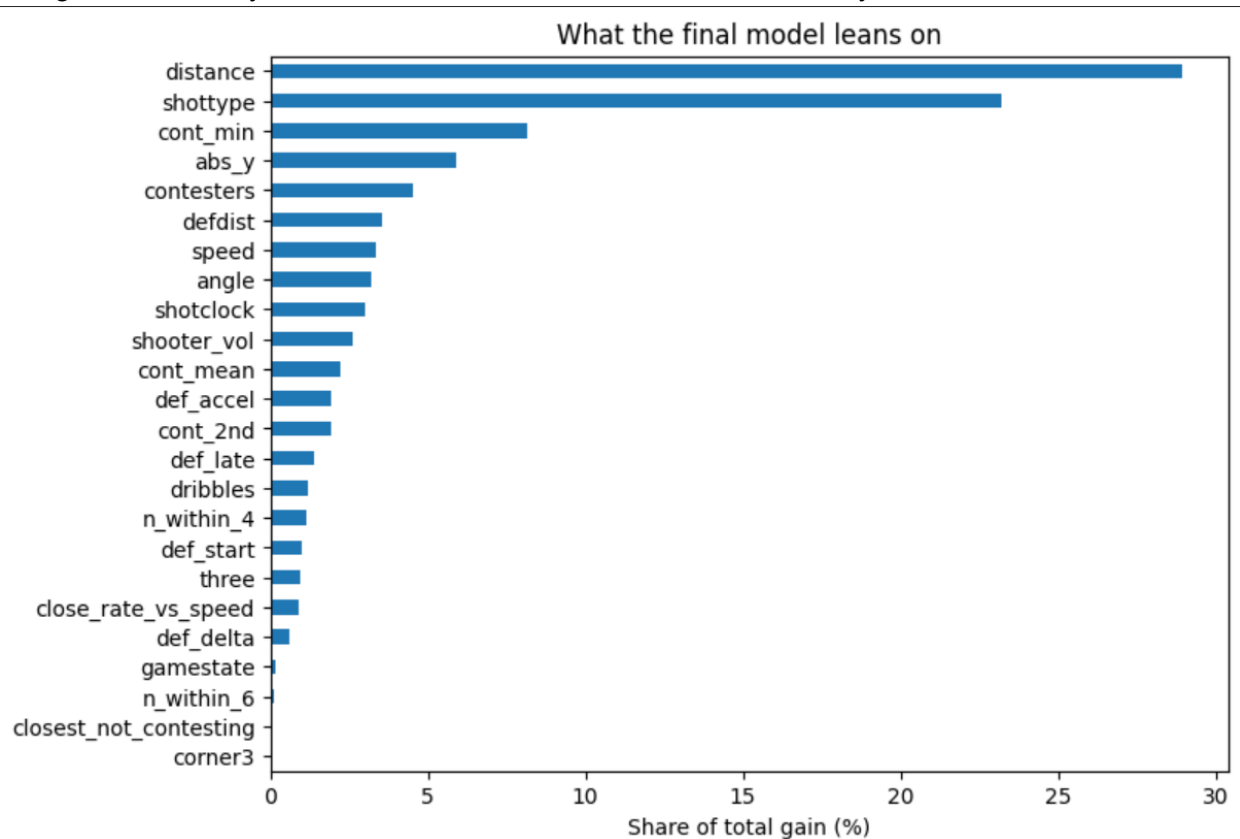


Figure 3-A

A gradient boosting decision tree model was chosen for this dataset as it captured the interactions between features best. The project's notebook showcases the various models that were tested but the idea was straightforward, where a constant baseline model was created to find our dataset baseline, next a logistic regression model with distance splines (a correction for the non-linear relationship distance has with make rate) was tested to then assume the basics

of a shot make probability model. From baseline to logistic regression log-loss was recorded at 0.6897 to 0.6380, where distance and shot type explain a majority of the shot quality and the rest meant trimming edges in respect to the data that was available. A linear model gives each feature one fixed effect, so it fails to represent combined states, averaging the effects instead of compounding them. Trees fit this better, because each split can carve out the combined states a shot may be in. Boosting specifically was chosen to solve a specific problem. The linear model's calibration curve (Figure 4-A) drifted above the diagonal at the top end at a predicted 0.72 make rate and the actual rate was about 0.77, and at 0.90 it was about 0.92 which was underselling the best shots; open dunks. A boosting model addresses these weaknesses, where the data metrics recorded offer much more with context than without, a key part of the game in the NBA, players are professional shooters and what differentiates each shot, dunk or layup is what is happening around that shot. What makes basketball interesting in terms of data is that a basket shot is nothing more than realistically a 50-50 probability, what changes those odds are that of the player if they are well equipped, the court if it is well defended, and the game state of how much pressure is on a given shot

Going back to the logistic model, where splines were brought in due to the idea of a straight line of shot distance against shot probability that fits an average slope does not adjust for what the game contains. The fact that make rate doesn't decline smoothly with distance, but falls sharply over the first four feet outside the restricted area, then stays roughly flat throughout the midrange, and drops again for the three pointers. For boosting this distance cliff meant virtually nothing as trees find their own threshold given a specific path.

Calibration is what carries the most weight for this specific project. The final model sits on the diagonal from 0.22 to 0.95 with a prediction range of 0.031 to 0.987. No post-hoc correction was applied, because the chart (Figure 4-B) showed none was needed. The reason this part is a heavy piece for the scope of the project is due to what we are producing, a probability surface. The target was not to guess a make or miss, but show that a model can be made that builds on many variables a single play in the NBA has, or simply put a model that measures shot quality.

With respect to log-loss and the final model, fold scores were 0.6277 and 0.6240, with a mean of 0.6258. Both folds were validated on a season the model had never trained on, which mirrors the situation the model faces when a third season is brought in. The model was given a limit of 3,000 trees to work with, the model used only about 160 and then stopped improving on the held out season meaning it found the signal fairly quickly. In the two-fold training, early stopping landed at 161 and 164 trees, so the final fit used 162. Sensible defaults were used for the model's hyperparameters. A few alternatives were tested at the end, but none of them returned anything meaningful enough to replace the defaults.

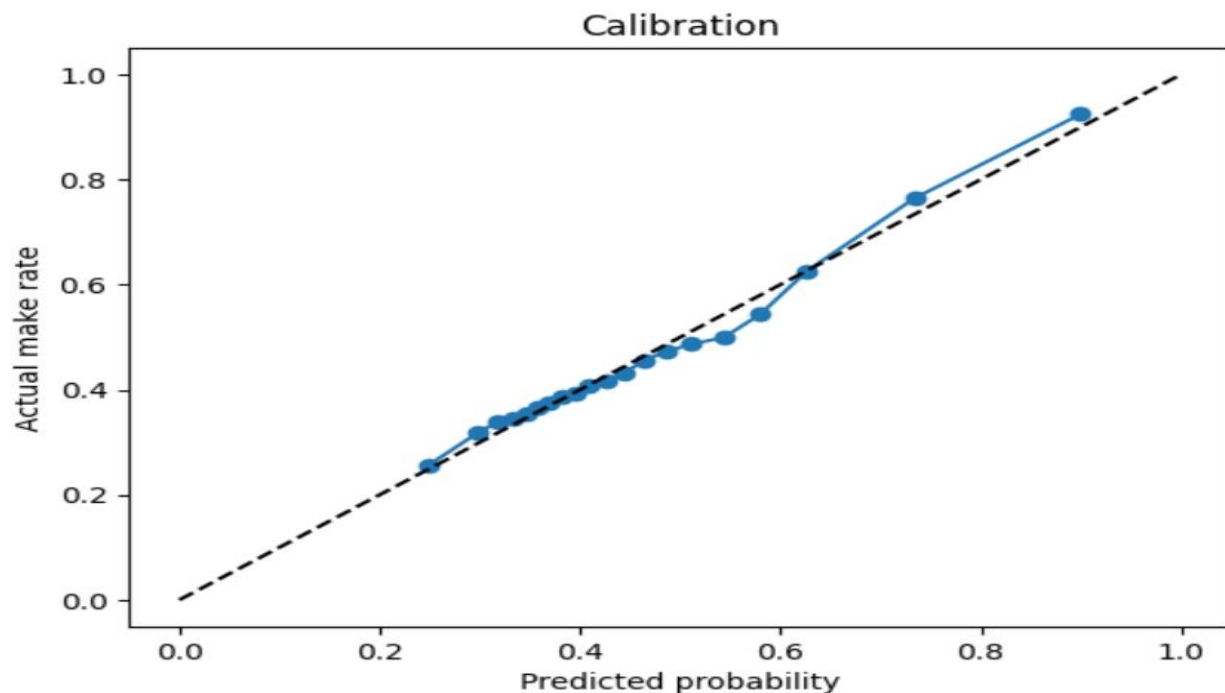


Figure 4-A Logistic Regression Model Calibration [prediction range: 0.021-0.971]

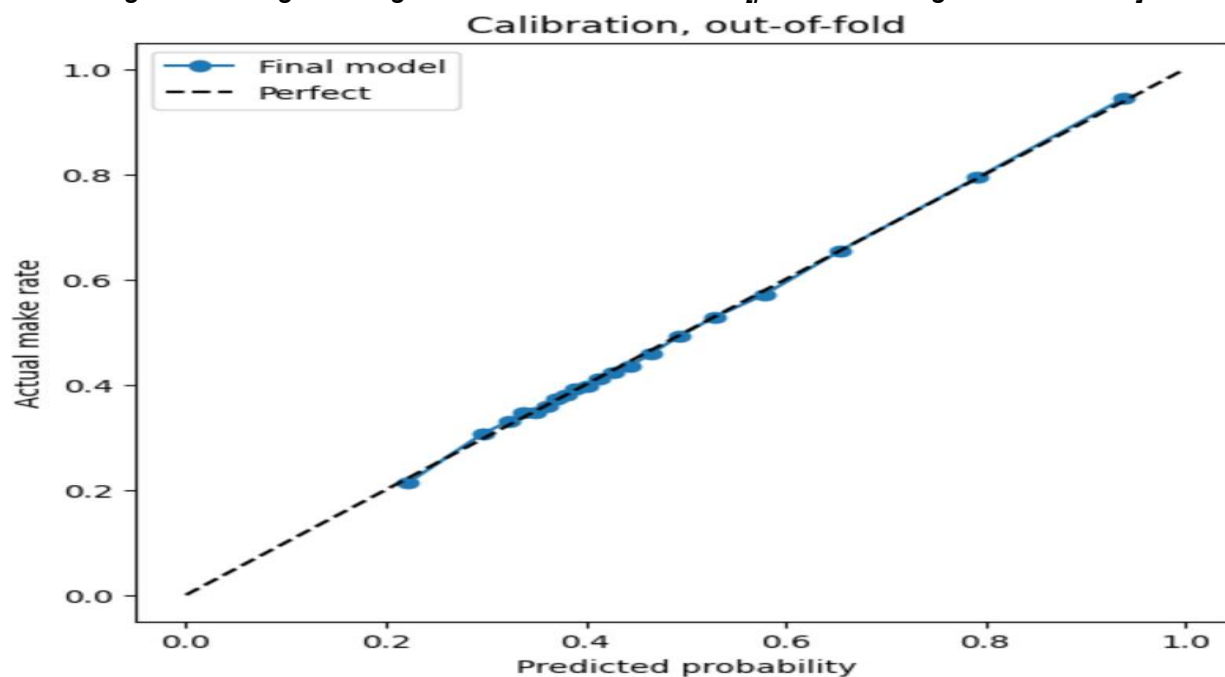


Figure 4-B Gradient Boosting Model(LightGBM) Calibration [0.031-0.99]

I tested eight ideas and kept three. The rejections took as much work as the additions and I believe they say more about the model than the features that survived. Before putting a number and weight to comparisons, the base model was run twice early on, returning log-loss of 0.6301 and 0.6296. With everything else kept identical, that set the threshold at 0.0005, and anything under it is essentially run-to-run randomness.

Shooter identity was mentioned early in the write up, where the conventional approach was to obtain the career make rate within the dataset for a player, I threw it out because a make rate is built from the outcome the model is trying to predict, which is exactly the kind of feature that leaks. What I was looking for was how often the specific player shoots, and volume answers what mattered. Volume skips over the idea of player skill, player shot-percentage, instead it tackles the problem from another perspective which is a player that takes 1,000 shots in the season is a shooter whether that be offensive game-plan or player profile. This feature turned out to be leak proof by construction which I had not fully appreciated when I proposed it. A count contains no outcome information so you can count a player's shots without knowing whether any went in, so there is nothing to compute out-of-fold and nothing to shrink. It gained 0.0011 (log-loss) from a single column.

The make rate version was still tested rather than assumed, and when added to the model it recorded a log-loss of 0.6242, the best number the project had seen. That number was wrong to follow, because the skill value was computed once from the whole training set, so during validation on season B the model looked up a number that already contained season B's makes and misses. The feature had seen, indirectly, the answers to the shots it was being scored on. Rebuilt inside each fold so that it only used the season being trained on, it produced a score of 0.6253, which is worse than the 0.6251 I got without the feature entirely. The improvement was entirely leakage. I had rejected this feature on judgement before any of it was measured, and the tests were run to confirm that judgement rather than to form it.

Defender behavior was the angle I most wanted to focus on, based on my own intuition about the game and what the dataset contained. I pushed on this point hardest since that was what the data contained and the fact that defenders make judgements in real time, their reaction reflects their own read of the shot quality, made by people who watch these players every day. This idea is originally seen in one of the first tables of the notebook that presented the make rate differences of large close-ins, meaning the shooter was more open initially and short close-ins meaning the shooter was guarded closely all the way to the start of the shot. A deeper translation points to defender judgement of the shot, a hard close in, in both the sense of covering a lot of ground or the defender speed represented a defender's judgement that a shot was of good quality with a make rate of (0.496) and shots where a defender peeled off went in least at (0.433). The [def_delta] is the change in defender distance, so it is mechanically tied to where the defender started: someone beginning twenty feet away can close a lot, someone beginning two feet away cannot. The correlation between them is -0.823, so "closed hard" is loosely code for "shooter was open". Then, controlling for defender distance at release, the effect survived. Within a fixed amount of space, closing behavior still moved make rate by 5 to 14 points, and the sign flipped for defenders already 5 or more feet away. That flip made sense to me; at close range a hard closer is arriving late at a look the shooter already has, while at longer range he is a genuine contest on a shot the shooter has time to see coming. The signal was real and separable, although the model recorded it as only 0.6% of gain in terms of what it leaned on. The honest conclusion is that the defense's judgement is mostly redundant with the shot itself once the model knows it is a dunk from four feet with one defender nearby, it can already infer most of what the defender's movement was telling me. The feature block of defender geometry brought in a total of 0.0017 gain the same as the three location features, where most of that came from spacing rather than motion. Meaning my biggest angle on the

data turned out to be a smaller piece than expected, nonetheless worth pursuing and investigating.

The features that stayed in but that the model proved redundant were [corner3], [three], and [closest_not_contesting] all at 0.0% or near it simply because the other features implicitly had these covered. Again the feature importance was only what the model used, and not what matters in basketball, an important distinction that influenced many decisions for this model.

Shot creation was another feature I initially proposed, where I expected dribble count to encode whether a shot was created or received, basically if it was a player or the offense manufactured a shot, the gain for this feature turned out to be 0. Thinking about it afterward made me realize that dribble count is a bad proxy for something the model can already see directly, where openness is measured by the defender distance and number of contesters. Trying to look at dribble count from another perspective, I tried the interaction version of pairing dribbles with what the defender did, in the theory that “dribbled and the defender fell behind” would describe the separation a shooter made themselves, but the gain was below the noise threshold and it was thrown out once again. The explanation was most likely due to gradient boosting building interactions as such on its own, splitting twice in the same branch where it was most likely another redundant feature.

Another front I tested was the calendar, where I think I had the right instinct but the wrong mechanism. I argued that the calendar matters, that October involves rookies fighting for a role and April or late season would involve teams pushing to win. The numbers proved the theory only half right, where the make rate was 0.4490 in October, and 0.4617 in April, but when split by season, October is low in both seasons (0.4454, 0.4515). In January the two seasons split to (0.4647, 0.4522), and in April the direction reversed entirely between them. The conclusion I came to was that since these are regular season games only, contending teams may run a different lineup in April for player rest and that eliminated teams may just be running deep bench lineups, meaning the population of shooters changes month to month. I believe this signal is compositional rather than temporal, where different kind of data and metrics would better suit this angle.

After numerous tests running trivial ideas, I came to the final conclusion that I reached a point of diminishing returns and had a solid model that captured the data and my ideas cleanly.

Biases in the data that were acknowledged included volume conflating skill with role, where shooter volume partly says how good this player is for the team, and partly says only that he is the one taking the shots on a low-percentage team. A high volume shooter on a weaker team is different from a strong role-player on a stacked team, same ability but the model cannot distinguish between them. Another bias is that the data covers only two seasons, which is thin for training a model that has to hold up on a new one. A further limit is that the dataset only contains shots that were actually taken, so a player who passed up an open look never appears, and every feature is conditioned on someone having decided to shoot.

What I would do with more data is continue digging into defender judgement. The data given only covers the closest defender's distance over one second and a count of contesters on the shot. It does not have where the other nine players were, what defense was being run, what play the offense ran, or where the ball came from. Defender commitment is a downstream effect of schemes and rotations, so the cause is invisible at this point. Additional data such as full player tracking, with all ten positions over the possession, would show how the defense is acting

and reacting and give cause as well as effect. For core shot measurements, shooter release mechanics would provide more on the shot itself where jumpers all fall into one category, but a stepback three is different from a set shot and a fadeaway. Another is possession context, where I started with my root idea in how I wanted to model the data, the thesis was that the offense's job is to move the defense, but that motion is not observable in this data, so I ended up pivoting toward the defense's effect on how the offense moves. In general I would really like to explore those ideas the most, since in sports at this level it is very dependent on gameplans compared to raw athlete talent. Lastly, more seasons would make shooter features stronger for the models, providing more for individual profiles, but also the weakest addition of data in my eyes. Given the current data I would not look to do more for my model, as I feel it is the data that limits my exploration, of course more tests could be done with more time but in the end I feel confident with my model given everything that was available. For improvement I wouldn't seek more time, but more data, or more specific data, to fully explore my defender cause and effect thesis.